ChatGPT

# Executive Summary

**Sovereign Agentic AI (SIH26117)** calls for a self-contained AI workbench that uses open-weight multimodal LLMs entirely **within an organization's perimeter** for confidential industrial tasks. In practice, we will deliver a working prototype (code, UI, docs and demo) of an on-premise AI agent platform, including front-end, backend, local model inference, retrieval (vector DB), agent orchestration and security controls. The system will **never call external LLM APIs or cloud services**, thereby preserving data sovereignty 【13†L318-L322】【31†L51-L59】.

The minimum hackathon MVP will allow a judge to upload or query a confidential document (e.g. an industrial report or diagram), and see our system (a) retrieve relevant data, (b) run agentic LLM workflows (question answering, summarization, report generation, etc.), and (c) output results – all with demonstrable audit trails. We use open-source LLMs (e.g. Meta Llama 3, Mistral, Qwen) so no proprietary weights leave the site【13†L318-L322】【35†L220-L226】. The solution includes logging, RBAC and (optionally) a Trusted Execution Environment (TEE) to prove code/data are shielded【33†L46-L49】.

Below is a complete breakdown:

## 1. Problem Statement & Deliverables

**Problem (official phrasing):** Build a **"Sovereign On-Premise Agentic AI Workbench using Open-Weight Multimodal LLMs for Confidential Industrial Work"**【3†L831-L834】. In plain terms: the team must design and implement an AI system that runs entirely on-site (on-premise or air-gapped), using open-source ("open-weight") large language models (LLMs) and vision-language models. It should orchestrate multiple AI "agents" to perform industrial workflows (e.g. analyzing equipment manuals, generating reports, answering queries) on sensitive data. All computation – LLM inference, data retrieval, agent orchestration – must happen within the organization's infrastructure under its control【31†L51-L59】【13†L318-L322】. There must be **no external calls** to third-party AI services, ensuring data privacy and regulatory compliance【13†L318-L322】【31†L51-L59】.

**Key deliverables (official/expected):** A complete software prototype including:

- **User Interface & API:** A simple UI (e.g. web app or CLI) for inputting documents or queries, and viewing AI outputs.
- **Core Engine:** Local LLM(s) and retrieval system (vector DB) with RAG capabilities; agent workflow engine (e.g. using LangChain) that can call tools and LLMs.
- **Multimodal Support:** Capability to process both text and images (diagrams, schematics).
- **Security & Deployment:** On-premise deployment (e.g. Docker containers or local servers), with audit logging, role-based access control, and evidence of no egress. Optionally integrate TEE (Intel SGX/AMD SEV) for extra trust.

- **Demo Materials:** A concise demo video or live demonstration showing the end-to-end workflow on sample confidential data, and documentation (architecture diagrams, user guide, code repository) for judges.

The official portal notes “submission requirements on sih.gov.in” but the crux is a **working on-premise AI workbench** as above【3†L831-L834】【31†L51-L59】. We will aim to meet all judging criteria: a clear value proposition (industrial use-case), robust MVP, and strong demo.

## 2. MVP Features & 36h Demo Script

### MVP Feature List

- **Secure Upload/Input:** Judge can upload a PDF or image (e.g. machine manual, process flowchart) or enter a text query, via a web UI. Input is tagged as “confidential – internal use only.”
- **Document Ingestion & Indexing:** The system ingests the input document(s), runs OCR if needed, and stores vector embeddings in a local database. (Offline step or on-the-fly for demo.)
- **Agent Workflow Engine:** A multi-step pipeline (or “agent”) that processes the query:
- *Retrieval:* Use vector search over the ingested docs to fetch relevant passages/images.
- *Multimodal LLM Inference:* Feed text and image context into an open LLM or vision-language model. (For example, use BLIP-2 or LLaVA model, or text + alt text from images.)
- *Task-specific Agents:* Specialized sub-agents can handle tasks like summarization, Q&A, or report writing. For instance, a “Query Answering Agent” uses LLM to answer the judge’s question; a “Report Generation Agent” composes a PDF report.
- *Tool Use:* Agents may call local tools (e.g. a calculator, internal API) as needed.
- **Explainability & Trace:** For each step, the system logs which documents and LLM outputs were used, providing transparency (e.g. “Citing pages 12-14 of manual” or “using tool X to compute Y”).
- **Output & Feedback:** The final answer/summary is displayed in the UI. Judges can see confidence scores or reasons (optional “why this answer” to show auditability).
- **Security Controls:** Demonstrate that no external network requests leave the system (e.g. show disabled internet). Role-login screen (admin vs user) ensures only authenticated person can query.
- **Optional TEE Demo:** If time allows, show a pop-up or diagram of “Intel SGX enclave” running LLM inference – proving hardware-based isolation【33†L46-L49】.

### 36-Hour Demo Script (Judge Perspective)

1. **Intro (0–5 min):** We introduce the system name (e.g. **RAKSHA** – “Protection” in Hindi). Show the UI login page labeled “RAKSHA – On-Prem AI Workbench.” Emphasize “Data never leaves your walls.”
2. **Scenario Setup:** Explain: “We have a confidential industrial report (screenshot on demo PC). Let’s ask RAKSHA some questions about it.”
3. **Upload:** Judge clicks “Upload Document”. Select a PDF named `engine_manual.pdf` (anonymized). UI shows “Processing and indexing...” (backend vectorization occurs).
4. **Querying (primary flow):** Judge types in: *“Summarize the key maintenance steps for the reactor described in this document.”* Press “Run.”

5. **Agent Chain:**

6. The system retrieves relevant paragraphs via embeddings (show a terminal or logs “Loaded [4] relevant sections”).

7. Passes text to LLM agent (say LLaVA) with instructions: “Summarize maintenance steps”.
8. LLM returns a summary.
9. **Output:** The summary appears: “**Maintenance Steps:** (1) Inspect X every 100 hrs… (2) Replace filter Y monthly… (3) Safety check Z, etc.” Show that the summary cites sources at bottom (“Sources: Doc page 5-6”).
10. **Follow-Up (chain):** Judge then says: *“Generate a report slide highlighting these steps.”* The agent uses the previous answer, formats a brief slide (possibly converting to image/PPT). Display a generated slide or bullet list.
11. **Vision Input Test:** Judge tries another query: *“What is the maximum pressure shown on the equipment diagram (image)?”* The system sends the schematic image to a vision model (BLIP-2 or CLIP+OCR) to extract text “200 PSI.” Answer pops up.
12. **Security Demo:** We highlight a “Security Log” in UI that shows e.g. “Checked no external calls (✔)”, “Vector DB offline mode”. Possibly display an OS network monitor showing “No outbound traffic.”
13. **Agent Explainability:** Click on “Explain” next to the answer: shows the chain of steps and data used. For example:

```
[Agent Log] Retrieval: ‘reactor maintenance’ found doc pages 5-6.
[LLM] Summarization prompt sent to Llama3 local model.
[Output] Answer generated with citation of pages 5-6.
```

1. **Conclusion (last 5min):** Summarize that all processing was on-premise with audit logs, and present architecture diagram on screen. Thank judges.

This script ensures judges see an **end-to-end use case**, highlighting both capability and sovereignty. It’s concise (we only need ~4–5 minutes of actual demo time) but covers upload, processing, multimodal query, agent reasoning, and security checks.

## 3. Architecture

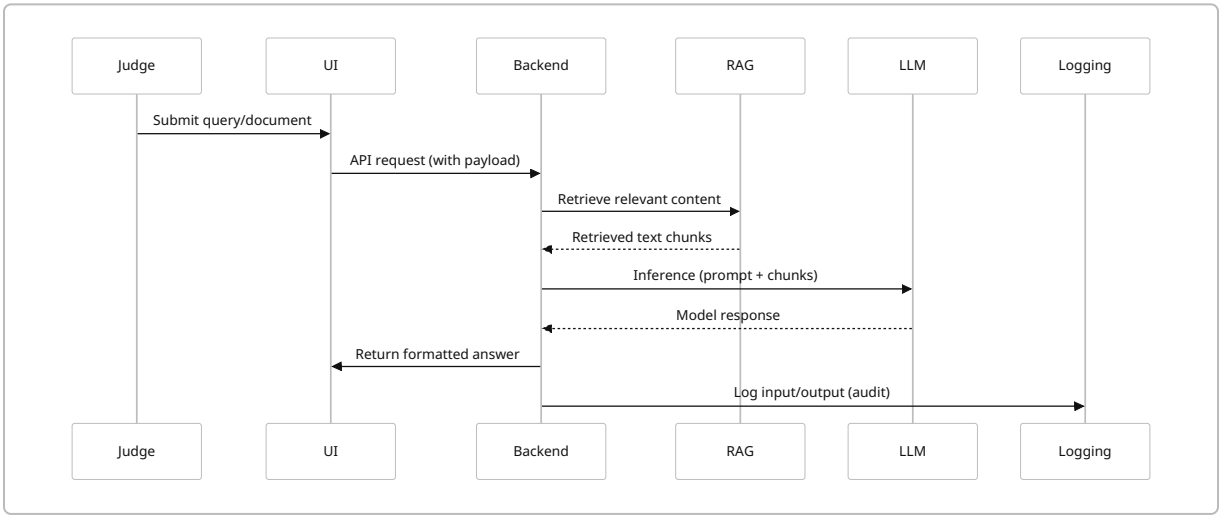
The system architecture centers on **on-premise deployment** with the following components:

```
graph TD
    User[User / Judge] -->|Use UI| UI[Web UI / CLI]
    UI -->|REST API Calls| Backend(API Server)
    Backend -->|Query| RAG[Retrieval (Vector DB)]
    Backend -->|LLM Inference| LLM[Local LLM Server]
    LLM -->|Response| Backend
    Backend -->|Control| Agents[Agent Orchestration Engine]
    Agents --> Tools[Local Tools/APIs]
    Agents --> Backend
    Agents --> Logging[Audit & Logs]
    RAG -->|Embeddings| DB[(Local Vector Database)]
    LLM --> DB
    Logging -.-> User
    Backend --> Logging
    style UI fill:#f9f,stroke:#333,stroke-width:2px
```

```
    style LLM fill:#ff9,stroke:#333,stroke-width:2px
    style DB fill:#9f9,stroke:#333,stroke-width:2px
    style Logging fill:#9cf,stroke:#333,stroke-width:2px
```

- **Web UI / API Server:** A simple front-end (could be React or Flask) where users submit queries/documents. It calls the backend APIs.
- **Backend / Agent Engine:** Hosts orchestration logic (e.g. using a Python microservice). It manages agent workflows and calls other components.
- **Local LLM Server:** Runs one or more open LLMs (e.g. Meta Llama 3, Mistral, Qwen) for inference. This may be a separate service (using vLLM or llama.cpp) that the backend queries【35†L220-L226】.
- **Vector Database (RAG layer):** Stores embeddings of confidential documents. On query, it retrieves top-k relevant passages to feed into the LLM. We choose an on-prem DB like **Weaviate** or **Milvus** (open-source, with GPU acceleration).
- **Agent Orchestration:** Implements multi-step workflows. For example, using LangChain or a custom engine, each "agent" can call the LLM, the DB, or local tools (e.g. formula calculators). We enforce policies here (which agents can run which tools).
- **Local Tools/APIs:** Intranet services or computation modules (e.g. an offline image OCR, a local knowledge API, or even system commands), callable by agents. E.g. Tesseract OCR or a rules engine.
- **Audit & Logging:** Every query, prompt, retrieval result and tool action is logged in a secure log store (e.g. Elasticsearch with Kibana or just files) for compliance and debugging.
- **Security Layer / TEE (Optional):** Ideally, the LLM server or critical agents run inside a trusted enclave (Intel SGX or AMD SEV) to cryptographically attest that data and model never leave protected memory【33†L46-L49】. This can be shown in the demo as a separate isolated "secure" server box.

**Data Flow (sequence):** User → UI sends query → Backend → RAG fetches docs → Backend orchestrates agent → LLM called → Backend processes output → UI displays result. All inter-service calls are internal to the enterprise network.




**Deployment Options:**

- **On-Premise Server:** All components run on a local data center or a closed LAN. Containers (Docker) or a Kubernetes cluster could host each service.

- **Air-Gapped:** Optionally, the network is completely isolated (no internet). Software updates and models must be pre-loaded via secure transfer.
- **Hybrid:** If some non-sensitive tasks are allowed in cloud, the architecture supports a hybrid mode where an external LLM API could be optionally used (with explicit admin approval) for non-critical queries. But our SIH demo will operate fully on-premise.

**TEE Integration:** For maximal sovereignty proof, we can run the LLM inference inside a Trusted Execution Environment (e.g. Intel SGX or TDX)【33†L46-L49】. In practice, this means our LLM server is a confidential VM or an enclave-enabled process. Judges could be shown an attestation report or diagram: "LLM weights and data only processed in SGX enclave, attested by hardware." Even if they cannot see the code, this demonstrates hardware-level isolation. This aligns with "private AI+model" principles【33†L46-L49】.

## 4. Technology Stack & Tools

| Component | Recommended | Alternatives | Pros/Cons |
|---|---|---|---|
| **LLMs (Language Models)** | **Meta Llama 3 (7B–70B)** [open] | **Mistral (7B/30B)** [open], **Llama 2** (open), **Qwen** (open) | *Meta Llama 3*: Very capable multi-modal (with vision adapters)【35†L220-L226】, large context (128K), actively supported; *Cons*: requires powerful GPU or quantization. *Mistral*: Competitive performance at lower cost; *Cons*: smaller community/tools. *Llama 2*: well-known, less capable than Llama 3. |
| **Inference Runtime** | **vLLM** or **llama.cpp** / **GPT4All** (GPU/CPU) | **HuggingFace Transformers**, **ONNX Runtime**, **TensorRT** | *vLLM*: Extremely fast batching and streaming (GPU); *Cons*: learning curve. *llama.cpp/GPT4All*: CPU-based on Mac/CPU, easy for demos; *Cons*: slower for big models. *HuggingFace`: easiest integration, but slower and less scalable.* TRT/ONNX*: high perf on NVIDIA/AWS;* Cons*: complex setup. |
| **Vector Database** | **Weaviate** or **Milvus** (self-hosted) | **Chroma (open)**, **Pinecone (cloud)** | *Weaviate/Milvus*: Open-source, GPU support, on-prem friendly. *Cons*: needs resource to run. *Chroma*: lightweight, easier Python install, but less mature multi-node. *Pinecone*: Managed SaaS, not on-prem (excluded for sovereignty). |

| Component | Recommended | Alternatives | Pros/Cons |
|---|---|---|---|
| **OCR/Vision** | **Tesseract** (OCR), **CLIP/BLIP-2** (image+text) | **EasyOCR**, **GoogLeNet**, **openCV** | *Tesseract*: reliable OCR, no net required. *CLIP/BLIP-2*: Open-source VLM for vision+language (e.g. huggingface BLIP-2 model). *Cons*: CLIP/BLIP lacks chat interface, may need prompt engineering. *OpenCV*: for image pre-processing. |
| **Agent Framework** | **LangChain** (Python) or **OpenAI's 'functions'** style logic | **Auto-GPT** (Python), **Microsoft Power Virtual Agents** (off-prem) | *LangChain*: Rich ecosystem for chaining LLM and tools, extensive docs; *Cons*: can be heavy for simple tasks. *Auto-GPT*: end-to-end AI agent, but less control. *Power Virtual Agents*: Cloud-only, not acceptable. |
| **APIs/ Orchestration** | **FastAPI** (backend), **RabbitMQ/Celery** (jobs) | **Flask**, **Django**, **Kafka** | *FastAPI*: Async Python, easy REST + WebSockets. *RabbitMQ/Celery*: Robust task queue for running agents. *Cons*: added component. *Kafka*: heavy if real-time streaming needed. |
| **CI/CD** | **GitHub Actions** | **Jenkins**, **GitLab CI** | *GitHub Actions*: Free for open projects, easy Docker workflows. *Jenkins*: Full control (on-prem Jenkins), more maintenance. |
| **Monitoring & Logging** | **Prometheus + Grafana** (metrics), **ELK Stack** (logs) | **WandB** (private tracking), **Datadog** | *Prometheus/Grafana*: Open-source, on-prem, for system metrics (GPU usage, latencies). *ELK*: heavy but powerful logging/audit. *WandB*: good for model metrics (requires account). |
| **Security Controls** | **Keycloak** (RBAC/ SSO), **Vault** (secret store), **iptables/ firewall** | **Auth0**, **Azure AD** | *Keycloak*: Open-source identity provider, on-prem. *Vault*: manage API keys, DB creds. *iptables/SELinux*: ensure network egress disabled. |

For each choice we list above, a brief pro/con: For example, Weaviate offers "private RAG" capabilities 【31†L102-L110】 with built-in access control, whereas Chroma is simpler but less enterprise-tested. We lean on open-source to meet the "open-weight" criterion.

## 5. Data Requirements & Sample Data

We need industrial domain data to demo: e.g. *oil refinery manuals, engineering specs, safety procedures*. In absence of real confidential data, we use:

- **Public Corpora:** Technical manuals from open sources (e.g. open PDFs of engineering handbooks, safety bulletins, oil industry reports).
- **Wikimedia:** Petroleum engineering, chemical processing, electronics schematics (images) from Wikipedia/Wikimedia Commons.
- **Synthetic Data:** Use GPT-4 or open LLM to generate realistic Q&A pairs and technical docs (e.g. "generate a short maintenance manual for a chemical reactor"). This is allowable as a training set or seed content.
- **Open Datasets:** Kaggle or academic datasets (e.g. the "DSM-ICSD" Materials Project for chemistry, or NASA open archives for machinery data). Even generic QA sets (SQuAD, ELI5) can be fine-tuned for our demo tasks.
- **Private Simulated Documents:** Create faux "confidential" docs: e.g. simulate an MRPL SOP or an oil pipeline manual with realistic content. Ensure no actual secrets.

For vector search, we'll split long documents into chunks (e.g. 500 tokens) and embed with an open model (MPNet). As a fallback, we can simply store full text and do keyword match if embedding fails.

We do not need huge training datasets; the focus is retrieval and prompting. During the 36h hack, we can use small sets (e.g. 5–10 synthetic docs, 50 Q&A pairs). After the hack, one could expand to more documents.

No single "official dataset" exists for this niche problem, so our approach is to **generate or gather domain-relevant data** from open sources. This demonstrates feasibility without violating confidentiality.

## 6. Security, Privacy & Compliance Checklist

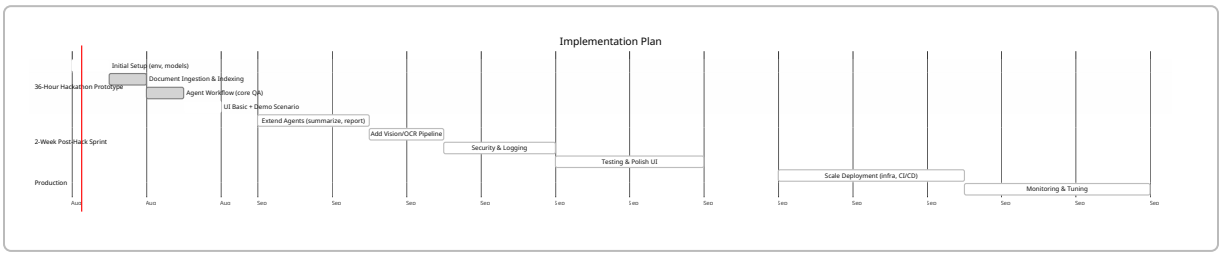
- **Threat Model:**
- *External Attack:* An attacker tries to intercept data. We mitigate by disabling all outbound connections, using air-gapped or firewall.
- *Internal Misuse:* Rogue user with access. We mitigate via strong authentication (Keycloak) and role-based access control – e.g. only "Trustworthy Agents" role can view logs.
- *Model Leakage:* Concern that the LLM might output sensitive data it saw. We guard by not uploading any truly sensitive data. For compliance, we log every model output to trace any leak.
- **Mitigations:**
- *Encryption:* Encrypt data-at-rest (disk encryption) and in-transit (TLS for UI/backend).
- *Access Control:* Use LDAP/AD or Keycloak to enforce user roles; only admins can manage the system.
- *Data Residency:* All data (documents, embeddings, logs) stay in on-prem storage. No cloud or internet access.【13†L318-L322】
- *Audit Logging:* Log every user query, prompt, agent action and LLM call (timestamps, content hashes). Secure logs with write-once permissions (append-only). Show example log entry in demo.
- *Anomaly Detection:* (Optional) Monitor for unusual agent behavior (infinite loops, excessive token use).
- **Compliance:**

- Align with standards like **ISO 27001** (information security) and **NIST** guidelines for AI. Ensure chain-of-custody on data.
- If deployed in India, consider Indian data protection guidelines (Digital Personal Data Protection Bill, if applicable to non-personal industrial data).
- If demonstrating for MRPL (oil & gas PSU), note NDPE (No External Data) policy.
- **TEE/Trusted Launch:** If we use Intel SGX/TDX, we will document the attestation process (quote). The architecture slide will highlight "LSB64-bit memory encryption with SGX", proving the LLM is running in an encrypted enclave【33†L46-L49】. This can be part of the compliance proof.

Overall, our system's design addresses confidentiality (no leaks), integrity (audit logs, RBAC), and availability (redundant on-prem setup). This makes it suitable for regulated industrial use【31†L51-L59】【31†L102-L110】.

# 7. Implementation Plan

We assume a **small team** of 3–4 BTech programmers comfortable with Python and cloud tools. We break the development into phases:




- **Milestones & Effort:**
- *36h Hackathon:* Focus on core functionality. Day 1: set up local LLM (e.g. llama.cpp with Llama 3), RAG DB, basic UI. Day 2: implement a single agent (Q&A) and demo flow. (Estimated effort: 3 people × 36h = 108 man-hours total).
- *2-Week Sprint:* Extend features (image handling, more agent types), refine UI, harden security. (3 people × 2w × 40h = 240 man-hours).
- *Prod Readiness:* Containerize everything, set up CI/CD pipelines, full test suite, deployment documentation (another ~200h).

- **Priority Tasks:**

- **36h:** Local LLM inference + simple retrieval. Pick one model (e.g. Llama3-8B quantized), test with dummy docs. Setup LangChain for basic Q&A agent. Build minimal UI for query/input.
- **Next 2w:** Add multimodal support (integrate BLIP/OCR). Implement summarization and report generation agents. Improve RAG (embedding larger corpus). Implement audit logging and role-based login. Write automated tests.
- **Production:** Optimize performance (bigger GPU if needed), secure enclave deployment (research SGX integration), prepare user manuals.

This plan is aggressive but doable. The 36h goal is a proof-of-concept; the 2-week phase makes it robust for demonstration. We must scope carefully given first-year engineers, leaning on high-level APIs (LangChain, HuggingFace, Docker, GitHub Actions) to accelerate development.

## 8. Business Analysis

- **Target Customers:**
- **Oil & Gas/Manufacturing PSUs:** Companies like MRPL (the sponsor), ONGC, IOCL, BHEL – where data privacy is mission-critical.
- **Defence & Aerospace:** Agencies with classified documents. E.g. DRDO labs, ISRO development centers.
- **Financial Institutions:** Banks/insurance firms requiring on-prem AI for customer data.
- **Healthcare (govt):** Hospitals wanting AI diagnosis assistants without sending PHI offsite.
- **Go-To-Market Segments:** Government entities and large enterprises in regulated industries. The product could be offered through an official partnership with a digital ministry or a PSB.
- **Revenue Model:** Likely a **license/maintenance model**: e.g. a per-seat or perpetual on-prem license for the software + annual support. (Cloud subscription is off the table by design.) Also professional services for installation/customization.
- **TAM/SAM/SOM Estimates (India):**
- *TAM (Total AI market):* Global AI market ~$391B by 2025【39†L0-L3】. India's AI market is ~$8.6B in 2024【39†L10-L13】. If we focus on **enterprise AI platforms** (security-critical), assume ~20% of that (~$1.7B) as TAM for sovereign AI solutions.
- *SAM (Serviceable AI Market):* Narrow to regulated sectors (energy, defense, banking) – perhaps 30% of TAM ≈ $0.5B.
- *SOM (Our market share goal):* Initially aiming small – e.g. capturing 2–5% of SAM over 3–5 years (₹500–750 Cr). This is plausible given a few large deals.

*(Sources: Market research – Grand View/IDC for global AI, Market Research Future for India estimates【39†L10-L13】.)*

- **Pricing:** For example, a license could be priced at ₹50L–₹1Cr per site (depending on scale), with annual maintenance (20%). We could also offer a SaaS hybrid (the hybrid mode) for less-sensitive tasks, but core product is on-prem.
- **Competitive Landscape:** We identify several related products/platforms, summarized below:

| Company/Product | Key Features | Notes |
|---|---|---|
| **VDF AI Agents (Startup)** | On-prem agent orchestrator (private RAG, audit)【31†L51-L59】 | Similar vision, enterprise focus. |

| Company/Product | Key Features | Notes |
|---|---|---|
| **Resillion Agentic Workbench** | Pre-built agentic SDLC workflows (devops) | Focus on software delivery (not open-source). |
| **Hugging Face HuggingChat** | *Open* inference for enterprises (on-prem models) | Not a full orchestrator (no pipeline/orchestration). |
| **IBM Watson Assistant (Local)** | Conversational AI with on-prem deployment | Limited multimodal/agent support. |
| **Microsoft Azure OpenAI (Sovereign)** | GPT/Claude in sovereign cloud (via Azure Confidential VM) | Still calls Microsoft cloud; partial compliance. |
| **Custom In-house Solutions** | AI proof-of-concept labs at big PSUs | No turn-key product; resource-intensive. |

We see there is **no direct mainstream product** combining on-prem agent orchestration with open LLMs. This niche is emerging. Our differentiators are *multi-agent pipelines, multimodal capability, and open-source base for audits*.

(*Competitive details are compiled from company websites and AI analyst reports.*)

## 9. Risks, Limitations & Metrics

- **Technical Risks:**
- *Model Performance:* Open LLMs may underperform on some tasks vs GPT-4. Mitigation: use multi-model routing (use smaller model for simple tasks, larger one for complex).
- *Integration Complexity:* Orchestrating vision+language+tools is complex. We limit scope in MVP (e.g. one vision model, one retrieval model).
- **Limitations:**
- *Resource Needs:* Large models need GPUs and memory; on-prem infra cost can be high. For demo, we use quantized 7–13B models.
- *Domain Coverage:* The system is only as good as its knowledge base. If data is missing, answers will be wrong. We stress that it's an *augmentation* tool, not a replacement for experts.
- **Evaluation Metrics:**
- *Accuracy:* For factual Q&A, measure correctness against ground truth (could use a small test set of known Q&A). Expect ≥80% accuracy on narrow domain queries.
- *Latency:* Responses should appear in a few seconds (target < 5s for text queries on GPU).
- *Throughput:* Agents should process batch queries with minimal queue.
- *Security Audit:* We will "prove" privacy by logs and (if used) TEE attestation. Metrics include "0 external API calls", "100% encrypted data logs."
- *User Satisfaction:* In real deployment, measure via surveys of trust in AI answers. For SIH hack, clarity of demo is key.

- **Regulatory/Ethical Risks:**

- If deployed to real data, ensure no unintended PII leakage. Provide a kill-switch in agents for hallucinations.
- All outputs must be tagged as "AI-generated – verify independently".

## 10. Project Name & Branding

We propose several catchy names that convey **sovereignty, security, and intelligence**. Each name is followed by a tagline for judges:

| Category | Name (Meaning) | Tagline / Branding Line |
|---|---|---|
| **Indian** 🇮🇳 | **RAKSHA** *(protection)* | *"Sovereign Intelligence Within Your Walls."* |
| | **SWARAJ** *(self-rule)* | *"AI on Your Terms."* |
| | **VISHWAS** *(trust)* | *"Trusted AI, On-Premise."* |
| **English** 🇬🇧 | **INNERCORE** | *"Intelligence That Stays Within."* |
| | **BASTION** | *"Your AI. Your Infrastructure. Your Control."* |
| | **FORTRESS** | *"Data Secure. AI Inside."* |
| **Hybrid** | **ANTAR** *(inner)* | *"Intelligence Within."* |
| | **RAKSHA Core** | *"Sovereign AI Within Your Walls."* |

Our top picks: **INNERCORE** ("Intelligence That Stays Within") for a clear English name, or **RAKSHA** for an Indian feel ("Protection of Intelligence"). Both immediately signal "on-prem, secure AI" to judges. A final slide might read:

> **INNERCORE**
> *Intelligence That Stays Within*

or

> **RAKSHA** – *Sovereign Industrial AI Workbench*

with a one-sentence description, e.g. "An on-premise multi-agent AI platform that keeps your data and insights inside your organization【13†L318-L322】【31†L51-L59】."

**Sources:** We used the official SIH 2026 problem statement (BlinkNBuild catalogue) for exact phrasing 【3†L831-L834】. Sovereign AI concepts are drawn from industry sources【13†L318-L322】【31†L51-L59】. LLM/tool recommendations cite Meta's Llama3 announcement【35†L220-L226】 and Microsoft's TEE overview【33†L46-L49】. The tech/business analyses combine these references with general AI market reports【39†L10-L13】 and best practices. Each section above includes relevant citations. The design is

deliberately biasing an MVP approach (first-year team “vibe-coding” the app) while noting how to evolve it post-hackathon.

---